Module 3

Tutorial 4
Introduction to Machine Learning

Marirena Bafaloukou

Tutorial 4 · Workshop, 10:30–17:00
Follow along in Intro_toML.ipynb



UK Dementia Research Institute

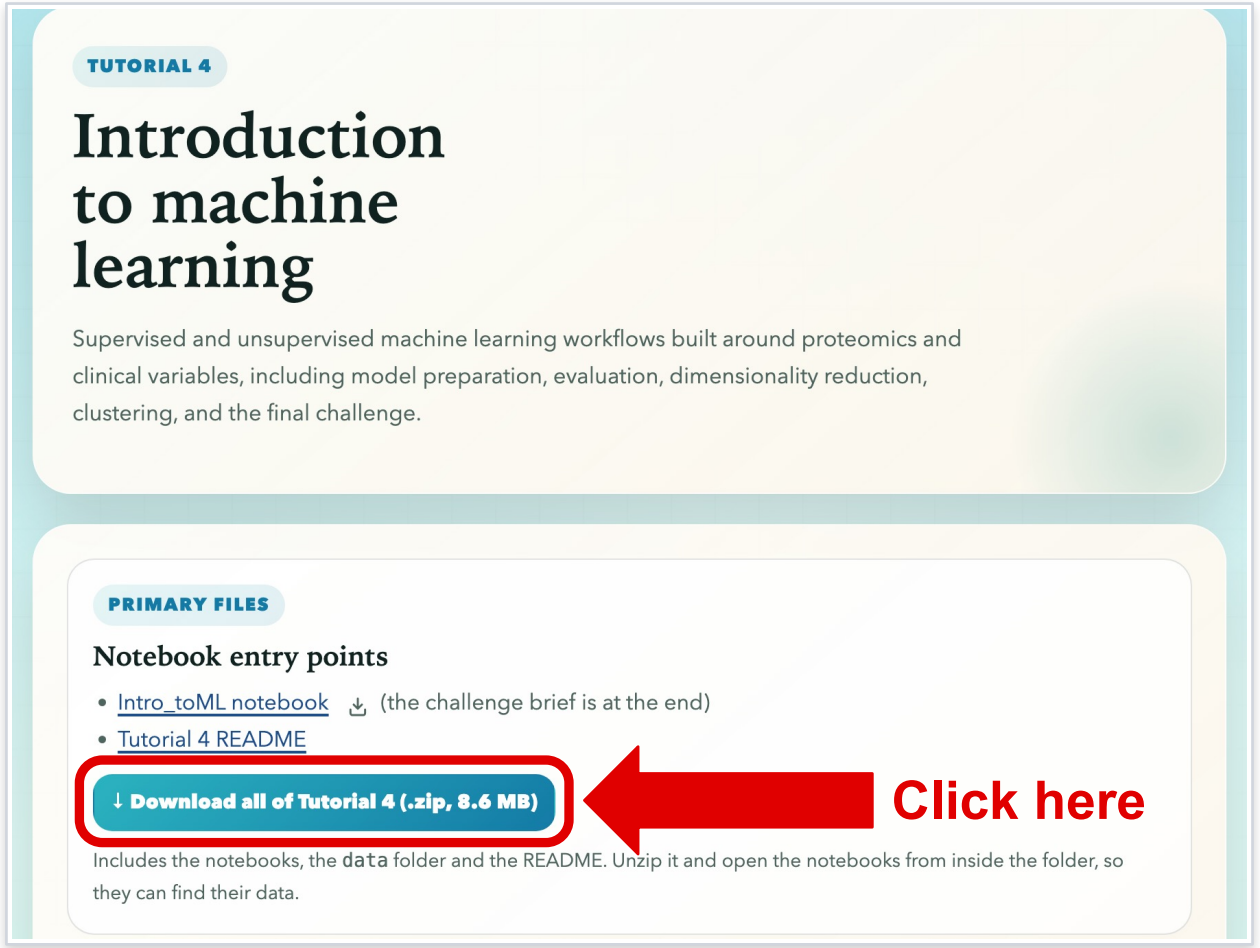
Before we start: get the materials

1. Go to the course website, **Tutorial 4**: sandoretal.github.io/Module3_2026

   Or scan it to open the Tutorial 4 page on your phone

2. Click **Download all of Tutorial 4 (.zip)**

3. **Unzip** it, and open **Intro_toML.ipynb** from inside the folder, so it can find its data



UK Dementia Research Institute

Today

📍 **In your notebook: Contents**

	Part	Question · You will be able to
☐	**Task 1 · Regression**	**Can we predict tremor severity from proteins?** Build a model for a continuous outcome, read its coefficients and residuals, evaluate it with MAE, MSE and R^2
☐	**Task 2 · Classification**	**Can we predict Parkinson's disease from proteins?** Build a model for a category, evaluate it with a confusion matrix, accuracy, precision, recall and specificity
☐	**Task 3 · Unsupervised learning**	**What structure is hidden in the data, without labels?** Reduce dimensions with PCA and UMAP, read explained variance, cluster with k-means and choose k
☐	🏆 **Challenge**	**Can plasma proteins tell us how old the gut is?** Explore intestinal organ ageing on your own, using regression, classification, PCA and UMAP

scikit-learn: our machine learning toolbox

📍 **In your notebook: Task 1 › import sklearn**

scikit-learn (`sklearn`) is a Python **library**: we **import** ready-made functions and machine learning models from it, instead of writing them ourselves.



What we import today

```
from sklearn.model_selection import train_test_split    # split
from sklearn.preprocessing import StandardScaler        # scale
from sklearn.linear_model import LinearRegression       # Task 1
from sklearn.linear_model import LogisticRegression     # Task 2
from sklearn.decomposition import PCA                   # Task 3
from sklearn.cluster import KMeans                      # Task 3
from sklearn.metrics import r2_score                    # evaluate
```

Every model: the same 3 lines

1 choose a model

```
model = LinearRegression()
```

2 train on the training set

```
model.fit(X_train, y_train)
```

3 predict for new patients

```
model.predict(X_test)
```

💡 Only the first line changes between tasks: LinearRegression, LogisticRegression, PCA, KMeans.

Task 1

Regression

Can we predict tremor severity from proteins?



Task 1: predicting tremor severity

📍 **In your notebook: Task 1**

$$y = \gamma + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$$

The data

500 people (about 30% with PD)

Age, sex and **40 plasma proteins**

A tremor score from wearable sensors, **0 to 10**

The goal

Predict the **tremor score** (a number) from the proteins, age and sex

This is **regression**

The model

Linear regression: the equation above.

One coefficient per feature

Data: `data/Proteomics_tremor_PD.csv`

The steps in your notebook

📍 **In your notebook: Task 1**

$$y = \gamma + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$$

Every supervised model today follows the same recipe:

1	2	3	4	5	6	7
Prepare	**1. Split**	**2. Missing values**	**3. Scale**	**4. Train**	**5. Interpret**	**6. Evaluate**
Numbers as float, sex as 0/1; define X and y	80% train, 20% test	Check; drop or impute	Fit on train, transform test	Linear regression learns the coefficients	Read the learnt coefficients	Predict the test set: MAE, MSE, R^2

```
In [ ]: y_pred = model.predict(X_test_scaled)
        from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
        r2 = r2_score(y_test, y_pred)
```

The red step is where data leakage happens: anything you **learn** from data, learn it from the **training set only**.

Your turn: Task 1

🍎 **Your turn · work through these in your notebook**

⏱️ **Work time** 35 min left

📓 **Look for this heading:** Task 1: Predicting tremor severity using proteomics and demographic variables.

	Step	In your notebook
▶️ Run	Load the data and define the protein columns	Task 1
✏️ Exercise	Define the numerical and categorical features	Data preparation
▶️ Run	Set the number types; turn sex into 0/1 (get_dummies)	Data preparation
✏️ Exercise	Combine age, sex and the proteins into one list: features	Data preparation
▶️ Run	Define X and y; split the data 80/20	1. Split
✏️ Exercise	Fix the code to split the data 70–30	1. Split
▶️ Run	Missing values, scaling, training, the learnt equation	2. → 5.
✏️ Exercise	What do the coefficients tell us about tremor?	5. Interpreting
▶️ Run	Predictions, the plot and the metrics (MAE, MSE, R^2)	6. Evaluating

🛠️ **Common errors**

`FileNotFoundError`
→ Open the notebook from inside the unzipped folder

`SyntaxError: invalid syntax`
→ Finish the exercise line (features = …), then run

`NameError: name 'X' is not defined`
→ A cell was skipped: run the cells above

`KeyError: 'sex'`
→ get_dummies ran twice: restart the kernel and run all

1. Split: training and test sets

📍 **In your notebook: Task 1 › 1. Split**

$$y = \gamma + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$$

```
#You need to specify the test size. Usually in a simple train/test
split we split the dataset into 70 % training set and 30% (70-30)
test set or 80% training set and 20% (80-20) test set.
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, #we need to split both the X (features) and the y (target)
    test_size=0.2, #0.2 corresponds to 20% of the data will belong
to the test set.
    random_state=42  #You need to set a random_state as a specific
number for reproducibility purposes. If you change the number the
splitting will be done differently.
)
```

Fit on train, transform test, then train

📍 **In your notebook: Task 1 › 3. Scaling and 4. Training**

$$y = \gamma + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$$

```
scaler = StandardScaler() # First you need to define the scaler instance

X_train_scaled = scaler.fit_transform(X_train)  # It is important to fit and
transform the scaler on the training data

X_test_scaled = scaler.transform(X_test) # It is important to  only transform the
test data using the scaler

from sklearn.linear_model import LinearRegression #The sklearn library has ready to
import and use ML models. Today we will use linear regression.

model = LinearRegression() # we create the instance of our model

model.fit(X_train_scaled, y_train) # we are "fitting it" on our dataset. This
represents the training phase.
```

Fit the scaler on the training set, then only **transform** the test set. `model.fit` learns the coefficients.

Task 1 as a regression

📍 **In your notebook: Task 1**

intercept coefficients error

$$y = \gamma + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$$

tremor severity 40 proteins, age and sex

In Task 1, **y** is the tremor severity, and the **X**s are the proteins, age and sex.

Training finds the intercept **γ** and the coefficients **β** that make the errors **ε** as small as possible. Then we use them to predict.

Back to the formula: what your model learnt

📍 **In your notebook: Task 1 › 5. Interpreting our model**

y: the average tremor — **the 3 biggest coefficients β**

$$\text{tremor} = 4.64 + 0.79\,\text{GFAP} - 0.74\,\text{ITGAV} + 0.72\,\text{SNCA} + \ldots + \varepsilon$$

y: tremor severity — **X: the proteins they belong to**

GFAP has the biggest **β**: one standard deviation more GFAP goes with about **0.8** points more tremor, with everything else held constant.

ITGAV's **β** is negative: more ITGAV goes with **less** tremor. Age (**0.04**) and sex (**0.06**) hardly matter once the proteins are in.

From the notebook's "Learnt equation" output: 80/20 split, random_state=42, scaled features. Your numbers should match.

Recap: Task 1

📍 **In your notebook: Task 1**

$$y = \gamma + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$$

✅ What you did

Prepared X (proteins, age, sex) and y (tremor)

Split 80/20, scaled, trained a **linear regression**

Read the learnt equation; evaluated on the test set

💡 Key ideas

Fit on the training set, **transform** the test set: no leakage

A coefficient = change in tremor per **1 SD** of that feature

R^2 = share of the tremor variation the model explains

💬 Discuss

What R^2 did you get? Is that good for biology?

Which features have the largest coefficients?

What changed with the 70–30 split?

Today: Task 1 done ✅

📍 **In your notebook: Contents**

	Part	Question · You will be able to
✅	~~Task 1 · Regression~~	~~Can we predict tremor severity from proteins?~~ ~~Build a model for a continuous outcome, read its coefficients and residuals, evaluate it with MAE, MSE and R^2~~
👉	**Task 2 · Classification**	**Can we predict Parkinson's disease from proteins?** Build a model for a category, evaluate it with a confusion matrix, accuracy, precision, recall and specificity
☐	**Task 3 · Unsupervised learning**	**What structure is hidden in the data, without labels?** Reduce dimensions with PCA and UMAP, read explained variance, cluster with k-means and choose k
☐	🏆 **Challenge**	**Can plasma proteins tell us how old the gut is?** Explore intestinal organ ageing on your own, using regression, classification, PCA and UMAP

Task 2

Classification

Can we predict Parkinson's disease from proteins?



Task 2: predicting Parkinson's disease

📍 **In your notebook: Task 2**

The data

The **same** dataset as Task 1

Now **proteins only**

The goal

Predict the **diagnosis**: 0 = control, 1 = Parkinson's

A category, not a number: **classification**

The model

Logistic regression: predicts the probability of PD, then a class (above 0.5 → PD)

The same recipe, with three new steps

📍 **In your notebook: Task 2**

New: **repeated measures** and **class imbalance** decide how to split; **feature selection** uses the tests from the statistics session.

1	2	3	4	5	6	7
Explore	**1. Split**	**2. Missing values**	**3. Select features**	**4. Scale**	**5. Train**	**6. Evaluate**
Samples per patient, cases vs controls	Keep the class ratio (stratify)	Check	Welch's t-test + FDR, on the training set	Fit on train, transform test	Logistic regression	Confusion matrix and metrics

```
In [ ]: y_pred = model.predict(X_test_scaled)
        from sklearn.metrics import confusion_matrix
        cm = confusion_matrix(y_test, y_pred)
```

Train the classifier

📍 **In your notebook: Task 2 › 5. Training**

```
from sklearn.linear_model import LogisticRegression #The sklearn library has ready to import and use ML models. Today we will use logistic regression.

model = LogisticRegression() # we create the instance of our Logistic Regression model

model.fit(X_train_scaled, y_train) # we are "fitting it" on our dataset. This represents the training phase.
```

Your turn: Task 2

🍎 **Your turn · work through these in your notebook**

⏱️ **Work time** 45 min left

📓 **Look for this heading:** Task 2: Predicting disease status using proteomics

	Step	In your notebook
▶️ Run	Look at the label, the participants and the class counts	Explore before splitting
✏️ Exercise	Define the features (proteins only) and the target	Explore before splitting
✏️ Exercise	Split the data with stratify	1. Split
▶️ Run	Check for missing values	2. Missing values
▶️ Run	Make the training-set table; test normality (Shapiro–Wilk)	3. Selecting features
✏️ Exercise	Check the variances with Levene's test	3. Selecting features
▶️ Run	Welch's t-test with FDR correction	Statistical Test Selection
✏️ Exercise	Keep only the significant proteins (training and test sets)	3. Selecting features
▶️ Run	Optional: boxplots with significance stars	Significance stars
✏️ Exercise	Scale the training and test sets	4. Scaling
▶️ Run	Train the logistic regression; predict; plot the confusion matrix	5. Training · 6. Evaluating
✏️ Exercise	Read the confusion matrix; calculate the metrics by hand and with scikit-learn	Confusion matrix · Metrics
▶️ Run	Extract the coefficients	7. Understanding the model
✏️ Exercise	Match each protein to its coefficient	7. Understanding the model

🛠️ **Common errors**

`ValueError: At least one array required as input`
→ train_test_split() is empty: give it X and y

`SyntaxError: invalid syntax`
→ Finish the exercise line (X_train_filtered = …), then run

`ValueError: The feature names should match …`
→ Filter the test set with the same proteins

`NameError: name 'X_features' is not defined`
→ A cell was skipped: run the cells above

Lunch break

Save your notebook: the afternoon carries on from the same variables.
Back at 13:30 in CXGH 1.14.



Recap: Task 2

📍 **In your notebook: Task 2**

✅ What you did

Explored the data: one sample per patient, 350 controls vs 150 PD

Stratified split; selected proteins with Welch's t-test + FDR

Trained a **logistic regression**; read the confusion matrix

💡 Key ideas

Feature selection and scaling on the **training set only**

Imbalanced classes: **accuracy alone can mislead**

Sensitivity, specificity and precision each answer a different question

💬 Discuss

Which proteins were significant?

How many false negatives did you get?

Which metric matters most for a diagnosis, and why?

Recap: the confusion matrix

📍 **In your notebook: Task 2 › Confusion Matrix**

	Predicted control	Predicted PD
Actually control	True negative (TN)	False positive (FP)
Actually PD	False negative (FN)	True positive (TP)

Which mistake is worse in a diagnosis: a false positive, or a false negative?

Today: Tasks 1 and 2 done ✅

📍 **In your notebook: Contents**

	Part	Question · You will be able to
✅	~~Task 1 · Regression~~	~~Can we predict tremor severity from proteins?~~ ~~Build a model for a continuous outcome, read its coefficients and residuals, evaluate it with MAE, MSE and R^2~~
✅	~~Task 2 · Classification~~	~~Can we predict Parkinson's disease from proteins?~~ ~~Build a model for a category, evaluate it with a confusion matrix, accuracy, precision, recall and specificity~~
👉	**Task 3 · Unsupervised learning**	**What structure is hidden in the data, without labels?** Reduce dimensions with PCA and UMAP, read explained variance, cluster with k-means and choose k
☐	🏆 **Challenge**	**Can plasma proteins tell us how old the gut is?** Explore intestinal organ ageing on your own, using regression, classification, PCA and UMAP

Task 3

Unsupervised learning

What structure is hidden in the data, without labels?



Task 3: finding structure without labels

📍 **In your notebook: Task 3**

The data

A **different** dataset: 500 people, about 40% with PD

209 proteins, age, sex, tremor 0–4

The goal

Find the structure **without** using the labels

Use diagnosis, age and sex only afterwards, to interpret

The tools

PCA (linear) and **UMAP** (non-linear) to reduce dimensions

k-means to find clusters

Data: `data/DF_for_unsupervised.csv`

The steps in your notebook

📍 **In your notebook: Task 3**

1 Clean
Remove the constant proteins

2 Scale
Scale the proteins

3 PCA & UMAP
2D plots by diagnosis, age, sex

4 How many PCs?
Explained variance: keep 80%

5 k-means
Cluster in the PCA space

6 Choose k
Elbow and silhouette

7 Zoom in
Cluster PD patients only: subtypes

In [ ]:
```
pd_mask = (df["PD_diagnosis"] == 1).values
X_pca_pd = X_pca_filtered[pd_mask]
df_pca_pd = df_pca[pd_mask]
```

The core pattern: PCA, then k-means

📍 **In your notebook: Task 3**

```
# Fit PCA
pca = PCA(n_components=n_components_80)
pca.fit(X_scaled)
X_pca_filtered = pca.transform(X_scaled)

# n_init=10: run k-means 10 times from different random starting
points and keep the best result
kmeans = KMeans(n_clusters=2, random_state=0,
n_init=10).fit(X_pca_filtered)
cluster_labels = pd.Series(kmeans.labels_, index=df_pca.index,
name='cluster').astype('category')
```

Your turn: Task 3

🍎 **Your turn · work through these in your notebook**

⏱️ **Work time** — **35 min left**

📓 **Look for this heading:** Task 3

	Step	In your notebook
▶️ **Run**	Load the data and list the protein columns	**Task 3**
✏️ **Exercise**	Remove the constant proteins	**Task 3**
▶️ **Run**	Count the cases; scale; PCA and UMAP plots by diagnosis, age and sex	**Task 3**
✏️ **Exercise**	What do PC1, PC2 and the UMAP show? Change the random state; colour by tremor	**Task 3**
▶️ **Run**	Explained variance; keep the PCs for 80% of the variance	**Number of PCs**
✏️ **Exercise**	How many PCs capture 70% of the variance?	**Number of PCs**
▶️ **Run**	k-means: the elbow plot and the silhouette scores	**Choosing k**
✏️ **Exercise**	Plot the silhouette score against k	**Choosing k**
▶️ **Run**	Cluster with k = 2 and plot it	**Choosing k**
✏️ **Exercise**	What are the 2 clusters? Repeat with k = 4 and interpret	**Clustering**
▶️ **Run**	Keep only the Parkinson's patients	**Zooming in**
✏️ **Exercise**	Silhouette for k = 2–6; cluster with k = 4; plot tremor against PC1	**Zooming in**

🛠️ **Common errors**

`FileNotFoundError`
→ Open the notebook from inside the unzipped folder

`SyntaxError: invalid syntax`
→ Finish the exercise line (constant_proteins = …), then run

`ValueError: Number of labels is 1 …`
→ Silhouette needs k ≥ 2: start the range at 2

`NameError: name 'X_pca_filtered' is not defined`
→ Run the "How many PCs" cells first

Recap: Task 3

📍 **In your notebook: Task 3**

✅ What you did

Removed constant proteins and scaled the data

Projected with **PCA** and **UMAP**; kept the PCs for 80% of the variance

Clustered with **k-means**; chose k; zoomed in on PD

💡 Key ideas

Unsupervised: **no labels** while fitting, only to interpret

Methods find the **strongest** structure first (here: sex)

Choose k with the metrics **and** the clinical question

💬 Discuss

What did PC1 and PC2 represent?

Did the PCA and UMAP plots change with the random state?

What did the 4 clusters within PD correspond to?

Today: Tasks 1–3 done ✅

📍 **In your notebook: Contents**

	Part	Question · You will be able to
✅	~~Task 1 · Regression~~	~~Can we predict tremor severity from proteins?~~ ~~Build a model for a continuous outcome, read its coefficients and residuals, evaluate it with MAE, MSE and R^2~~
✅	~~Task 2 · Classification~~	~~Can we predict Parkinson's disease from proteins?~~ ~~Build a model for a category, evaluate it with a confusion matrix, accuracy, precision, recall and specificity~~
✅	~~Task 3 · Unsupervised learning~~	~~What structure is hidden in the data, without labels?~~ ~~Reduce dimensions with PCA and UMAP, read explained variance, cluster with k-means and choose k~~
👉	🏆 **Challenge**	**Can plasma proteins tell us how old the gut is?** Explore intestinal organ ageing on your own, using regression, classification, PCA and UMAP

What you learnt: the workflow and regression

📍 **In your notebook: Tasks 1–3**

1 Split first
80% train · 20% test

→

2 Fit on train
scaler, selection, PCA, UMAP

→

3 Apply to test
same fitted objects

→

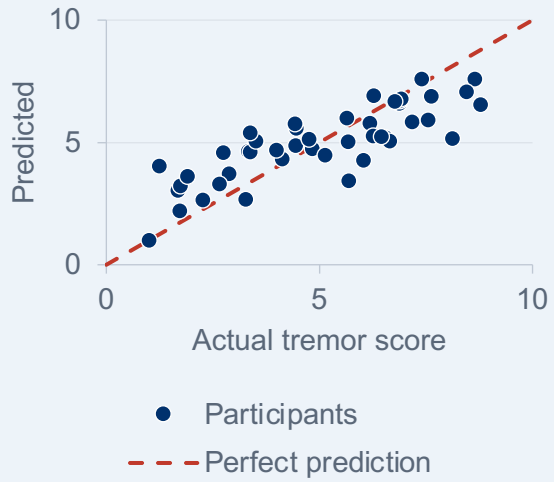
4 Train
e.g. linear or logistic regression

→

5 Evaluate
on the test set only

⚠️ **No leakage:** the test set is never used to fit anything, not even the scaler or the feature selection.

📈 Regression: predict a number (Task 1)

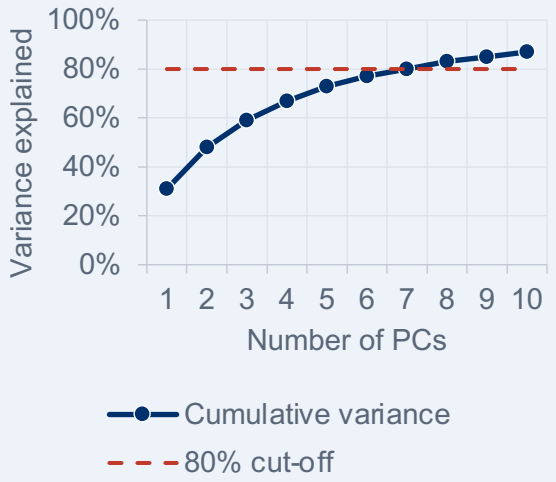


R^2
variation explained · higher = better

MAE
average error size · lower = better

MSE
squared errors: big misses count more

🧬 Choosing features (Tasks 2–3)



Statistical selection
test each protein on the training set; correct for multiple testing

PCA
keep the PCs for 80% of the variance

UMAP
non-linear; fix random_state

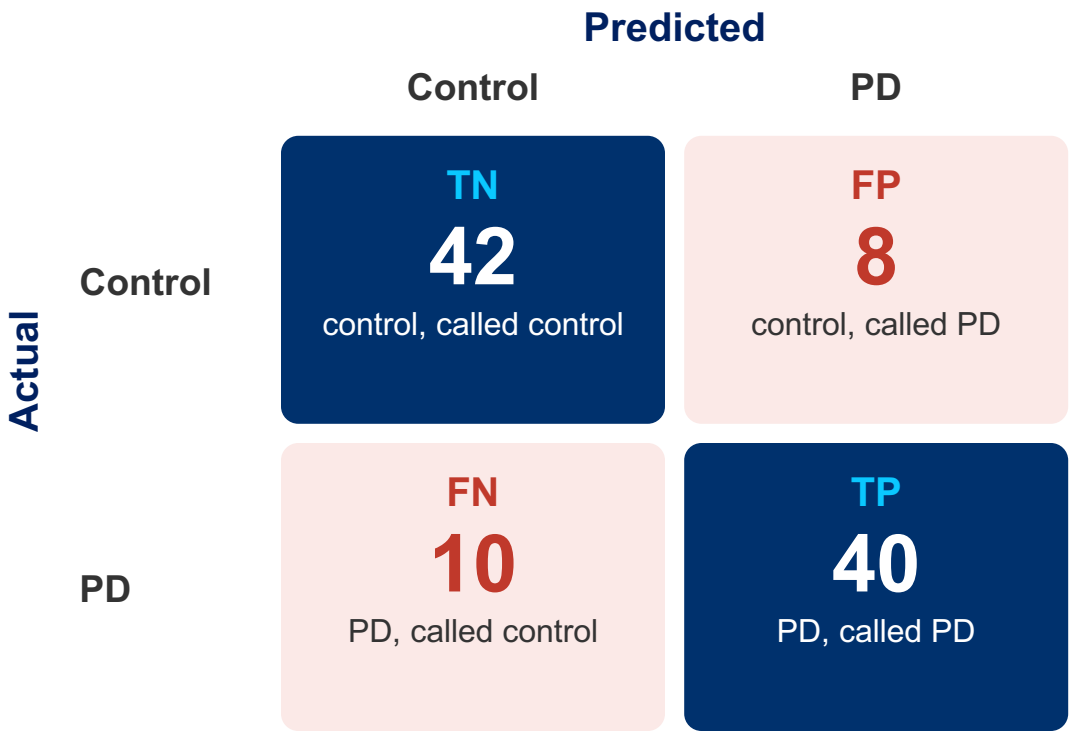
Illustrative data, not your results.

UK Dementia Research Institute

What you learnt: classification

📍 **In your notebook: Task 2 › Confusion matrix and metrics**

parkinson_diagnosis — **0 → Control** | **1 → Parkinson's (positive class)**



✓ **dark = correct** ✗ **red = errors** · illustrative counts

Accuracy (TP + TN) / all
How often is it right overall? **0.82** 82 / 100

Precision TP / (TP + FP)
Of those called PD, how many have PD? **0.83** 40 / 48

Recall (sensitivity) TP / (TP + FN)
Of the PD patients, how many did it find? **0.80** 40 / 50

Specificity TN / (TN + FP)
Of the controls, how many did it find? **0.84** 42 / 50

💡 **Check the class balance first:** if 70% are controls, always saying "control" already scores 0.70 accuracy.

🏆 Challenge

Intestinal organ ageing

Exploring intestinal organ ageing using plasma proteomics:
on your own, in the notebook



Challenge: intestinal organ ageing

📍 **In your notebook: 🏆 Challenge: Plasma Proteomics and Organ-Specific Aging**

- Last, you will work on a challenge by yourself, where you will be asked to explore **intestinal organ ageing using plasma proteomics**. You will work on this in the notebook: add your own cells below the challenge.
- Plasma proteins can estimate the **biological age of organs** (Oh et al., 2023); the gut may show early changes in Parkinson's (the "body-first" hypothesis).
  - **Explore:** organ age vs **constipation** (Wexner), **smell** (UPSIT) and **motor** symptoms (UPDRS III)
  - **Regression:** predict organ age from all proteins, significant proteins, PCA and UMAP features
  - **Classification:** young vs old organs, with logistic regression
  - **Report:** compare the feature sets in a table and plots

Everything you need

The full instructions are in the last section of **Intro_toML.ipynb**.

Use the steps from today: split, scale, select features on the **training set only**, train, evaluate.

📓 **Look for this heading in your notebook:**

🏆 Challenge: Plasma Proteomics and Organ-Specific Aging

Work time: the challenge

🍎 **Your turn · work in your notebook**

⏱️ **Work time**

90 min left

The bar shows the time left

Independent practical: **no solutions provided**. Finish Task 3 first if you need to. The teaching assistants are in the room.

Challenge roadmap: copy and adapt the code you already have

Challenge step	Copy and adapt from
☐ 1. Load the data, patient_id as index	Task 1 › the first cells (imports, pd.read_csv)
☐ 2. Organ age vs constipation, smell, UPDRS III	A correlation (scipy.stats) and a seaborn scatterplot, as in Task 3
☐ 3.1 Regression: split, scale, train, evaluate	Task 1 › 1. Split → 6. Evaluating (R^2, MAE, MSE)
☐ 3.2 Proteins correlated with organ age	Task 2 › 3. Selecting features: same loop, training set only
☐ 3.3 PCA (80% variance) and UMAP features	Task 3 › PCA & UMAP, How many PCs
☐ 4. Young vs old organs (< 0.5 / ≥ 0.5)	Task 2 › 5. Training → Metrics, confusion matrix
☐ 5–6. Comparison table, bar plot, discussion	New: put the metrics in a pd.DataFrame

🛠️ **Common errors**

`FileNotFoundError`
→ Run it from inside the tutorial folder

`KeyError: 'organ_age'`
→ The column is intestinal_organ_age: check df.columns

`NameError: name 'pd' is not defined`
→ Import your libraries at the top first

`ValueError: The feature names should match …`
→ Use the same columns for train and test

Your turn

Work through the exercises and the challenge in Intro_toML.ipynb.

The teaching assistants are in the room.

